of 26 cycles of the alphabet returns it back to its original positions; hence, each letter remains unchanged. Therefore, only shifts between 1 and 25 are meaningful in Caesar cipher, as a shift of 0 also leaves the text unchanged.

The Caesar cipher is a type of monoalphabetic substitution cipher, where each letter in the plaintext is consistently replaced by a letter from a fixed substitution alphabet. In monoalphabetic substitution ciphers, the same letter is always substituted by the same corresponding letter throughout the entire message. The security of the Caesar cipher relies on the attacker not knowing the shift value. However, as we have already observed, there are only 25 possible shifts, making it easy for an attacker to try all of them and quickly discover the plaintext. This method of attack is known as a brute-force attack. Because of this vulnerability, the Caesar cipher and similar monoalphabetic substitution ciphers are considered ineffective for secure communication. This limitation has led to the development of more advanced and harder-to-crack encryption methods, such as polyalphabetic substitution ciphers.

Vigenère cipher

Polyalphabetic substitution ciphers use multiple substitution alphabets to encrypt a message, making them more secure than monoalphabetic substitution ciphers. In this method, the same letter can be replaced by different letters depending on its position in the text, which reduces patterns and makes frequency analysis attacks more difficult. The Vigenère cipher (invented by Giovan Battista Bellaso in 1553 but misnamed after Blaise de Vigenère) is a well-known polyalphabetic cipher that uses a repeating key (a group of random characters) to shift letters by different amounts. Each letter in the key corresponds to a different Caesar cipher, and the shifts are applied in a cycle. This way, the same plaintext letter can be substituted by different ciphertext letters based on its position in the sequence, enhancing security.

Let us try to understand the working of the Vigenère cipher using the plaintext 'BCDEFBCDE' and the key 'BCD'. We will first define how letter values are assigned. In the Vigenère cipher, each letter corresponds to a numerical value as follows: A = 0, B = 1, C = 2, D = 3, E = 4, and so on up to Z = 25. Now, we will walk through the encryption using the key 'BCD'. The plaintext values (using A = 0, B = 1, etc) are B = 1, C = 2, D = 3, E = 4, F = 5, B = 1, C = 2, D = 3, and E = 4. Key values (repeated to match the length of the plaintext) are B = 1, C = 2, D = 3, B = 1, C = 2, D = 3, B = 1, C = 2, and D = 3. The encryption happens as follows. Ciphertext values are obtained as (plaintext + key) mod 26. You can understand the working of the modulus operation by recalling the example of a clock. The corresponding positions of the letters in the plaintext, key, and ciphertext are shown below.

```
B  C  D  E  F  B  C  D  E
B  C  D  B  C  D  B  C  D
---------------------------------
C  E  G  F  H  E  D  F  H
```

Now, let us go through the encryption process in detail. We get: (B+B) → (1+1)=2 → C, (C+C) → (2+2)=4 → E, (D+D) → (3+3)=6 → G, (E+B) → (4+1)=5 → F, (F+C) → (5+2)=7 → H, (B+D) → (1+3)=4 → E, (C+B) → (2+1)=3 → D, (D+C) → (3+2)=5 → F, and (E+D) → (4+3)=7 → H. Thus, the final ciphertext is CEGFHEDFH. Keep in mind that the first 'B' in the plaintext became a 'C' and the second 'B' became an 'E'. Similarly, the first 'E' in the cipher text is a substitution for the letter 'C' in the plaintext and the second 'E' is a substitution for the letter 'B' in the plaintext. This is different from the Caesar cipher. In this way, polyalphabetic substitution ciphers like Vigenère cipher are far stronger than a Caesar cipher.

Let us now implement the Vigenère cipher using SageMath. The program 'vigenere21.sagews', as shown below, implements a Vigenère cipher function that can both encrypt and decrypt text based on the given key and mode (line numbers are included for clarity). For simplicity, the program assumes that both the plaintext and the key consist solely of uppercase letters from the English alphabet, with no spaces or special characters allowed.

```
1.  def vigenere_cipher(data, key, mode):
2.      alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
3.      text = [ ]
4.      key_length = len(key)
5.      for i, char in enumerate(data):
6.          shift = alphabet.index(key[i % key_length]) * mode
7.          new_char = alphabet[(alphabet.index(char) + shift)
            % 26]
8.          text.append(new_char)
9.      return ''.join(text)
```

Now, let us go through the code line by line to understand how it works. Line 1 defines a function named *vigenere_cipher( )*, which takes three arguments: *data* (the text to be encrypted or decrypted), *key* (the string used for encryption or decryption), and *mode* (an integer that determines whether the function will encrypt or decrypt). In Line 2, a string containing all the uppercase letters of the English alphabet is assigned to the variable *alphabet*. This will be used to look up the positions of letters. Line 3 initialises an empty list named *text*, which will store the processed characters (either encrypted or decrypted) as the function iterates over the input data. Line 4 calculates the length of the key string and stores it in the variable *key_length*. This will help determine how to cycle through